

Octowize Platform Documentation

	1
1. Project Overview	1
2. Technology Stack	2
3. System Architecture	2
3.1 Interaction Flow	2
3.2 Architecture Diagram (Textual Format)	3
3.3 Authentication & Routing	3
4. Project Directory Structure	3
4.1 Backend	3
4.2 Frontend	4
5. Frontend Application	5
5.1 Page Flow	5
Onboarding Paths:	5
Other Pages:	5
5.2 Key Components	6
5.3 Supabase Integration Points	6
5.4 Chatbot Integration	6
6. Backend Application	7
6.1 FastAPI Structure	7
6.2 RAG Chatbot Engine (rag_engine.py)	7
6.3 Supabase Client Integration	8
7.1 Tables & Schemas	8
8. Chatbot Architecture	9
8.1 Gemini + PgVector Flow	9
8.2 Current Limitations	10
8.3 Future Scope	10
9. Authentication & Authorization	10
9.1 Authentication	10
9.2 Role-Based Access	11

1. Project Overview

Octowize is an AI-powered business onboarding platform designed to streamline the company incorporation process in the UAE. It supports entrepreneurs and investors in selecting the right jurisdiction (Freezone, Mainland, or Offshore), fulfilling regulatory requirements, and submitting essential business details — all through an interactive, step-by-step onboarding interface.

The platform integrates:

- **React** for a responsive, multi-step frontend.
- **FastAPI** to power a chatbot that answers legal and procedural queries using RAG (Retrieval-Augmented Generation) and Gemini API.
- **Supabase** for secure user authentication, data storage, and form submission tracking.

Octowize simplifies decision-making by combining regulatory guidance with automation, reducing the friction traditionally associated with UAE business setup.

2. Technology Stack

Frontend: ReactJS

Backend: FastAPI (Python)

Database & Auth: Supabase (PostgreSQL + Auth)

Vector Store: pgVector Database

Chatbot LLM: Gemini API (RAG-enabled)

Environment Management: `venv`, `.env`

Deployment: (e.g., Vercel, Render, etc. — to be filled)

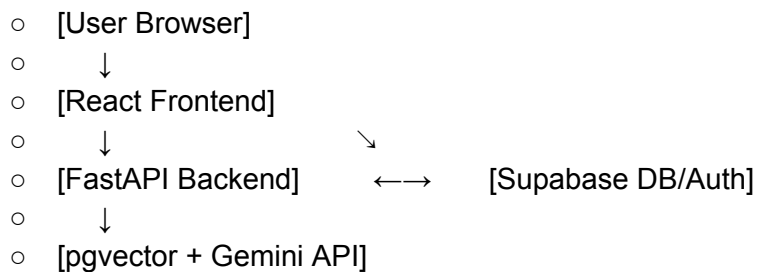
3. System Architecture

The Octowize platform follows a **modular full-stack architecture**, consisting of three main layers: **Frontend**, **Backend**, and **Database/LLM Services**.

3.1 Interaction Flow

1. **User** interacts with the React-based onboarding UI.
2. **Frontend** sends/receives data to/from:
 - **Supabase** (for auth, form data, business info)
 - **FastAPI backend** (for chatbot queries)
3. **FastAPI** backend processes chatbot queries by:
 - Rephrasing the question
 - Searching LanceDB for relevant documents
 - Calling **Gemini API** to generate a response
4. The **chatbot response** is returned and displayed in the frontend.
5. **Admin** interacts with React-based UI with privileges like viewing user info, downloading and Approving User document

3.2 Architecture Diagram (Textual Format)



3.3 Authentication & Routing

- Auth is managed by **Supabase**.
- Logged-in users are routed to **Mainland**, **Freezone**, or **Offshore** onboarding.
- Admins are directed to a separate dashboard.

4. Project Directory Structure

4.1 Backend

```
backend/
├── app/
│   ├── main.py           # FastAPI entry point
│   ├── api/routes.py     # Chatbot API endpoint(s)
│   ├── cors/config.py    # CORS configuration
│   ├── Freezone_Research/ # Static research content
│   ├── models/           # (Optional) Data models
│   ├── services/rag_engine.py # RAG-based chatbot logic
│   └── supabase/supabase_client.py # Supabase backend client
├── .env                 # Environment variables
└── requirements.txt      # Python dependencies
```

- `main.py`: Entry point
- `api/routes.py`: All backend endpoints
- `services/rag_engine.py`: Chatbot logic using RAG
- `supabase/supabase_client.py`: DB interaction
- `db/lancedb`: Local vector storage
- `.env`: Secrets and config
- `requirements.txt`: Python dependencies

4.2 Frontend

```
frontend/
├── public/              # Static files (HTML, icons)
├── src/
│   ├── assets/          # Logos, images, SVGs
│   └── components/      # Reusable UI components (chatbot, navbars, sidebars)
```

css/style.css	# Global styles
pages/	
freezezone/	# Freezezone onboarding steps
mainland/	# Mainland onboarding steps
offshore/	# Offshore onboarding steps
admin/	# Admin dashboard & user details
userlogin/	# User login and signup via Auth
general pages	# Homepage, Not Available, etc.
supabaseClient.js	# Frontend DB/auth config
App.js	# Main router & layout
index.js	# App bootstrap
.env	# Frontend environment config
package.json	# Project dependencies

- `pages/`: Route-based pages (Freezone, Mainland, Offshore)
- `components/`: Common components (chatbot, navbar, sidebar)
- `css/`: Global and page-specific styles
- `supabaseClient.js`: Supabase frontend config
- `App.js`: Router + global layout

5. Frontend Application

The frontend is built using **ReactJS**, designed around a **multi-step onboarding flow** with separate paths for Freezone, Mainland, and Offshore setups. It also includes a dynamic chatbot and admin interface.

5.1 Page Flow

Authentication Pages

- **SignupPage.jsx**

Allows new users to register using their name, email, and password.

- Implements Supabase Auth with email verification.
- On successful signup, user metadata is stored in Supabase Auth and a corresponding entry is inserted into the **Users** table.
- Triggers creation of a default **Onboarding** record for tracking the user's progress.
- Redirects to **signin** after showing a verification prompt.

- **SignInPage.jsx**

Enables users to sign in based on their role.

- **Regular Users:** Authenticated using Supabase Auth. If the user record doesn't exist in the **Users** table, it is inserted and onboarding is initialized.
- **Admins:** Bypasses Supabase Auth and checks the manually maintained **Admins** table. Admins are redirected to the admin dashboard upon successful match.
- Role is stored in **localStorage** for route-based access control.

- **AuthCallback.jsx**

Handles post-verification redirection after email confirmation.

- Retrieves the authenticated user via Supabase Auth.
- Ensures corresponding **Users** and **Onboarding** records exist.
- Redirects to onboarding flow or dashboard based on payment and submission status.

Onboarding Paths:

- **Freezone:** 14-step form-based onboarding (e.g., industry selection, visa needs, office preference)
- **Mainland:** Form inputs like legal structure, employee count, and business name
- **Offshore:** Simpler flow with 2 steps

Other Pages:

- `Homepage.jsx`: Landing page
- `notavailable.jsx`: Shown when a feature is under development
- `maincontainer.jsx`: Controls overall routing and layout

Admin Pages (`src/pages/admin/`):

- **`AdminDashboard.jsx`** –
Displays a centralized dashboard for admins, listing all users who have completed the onboarding (`submitted = true`). The dashboard is divided into three sections:
 - **Freezone Users:**
Shows user name, industry, activity, and freezone, with action buttons to view full details.
 - **Mainland Users:**
Displays business name and legal structure, with a detail view option.
 - **Offshore Users:**
Displays business name and jurisdiction, with a detail view option.
- The data is fetched from multiple tables including `Onboarding`, `Users`, `Industry`, `Activities`, `Freezones`, `MainlandUserInfo`, and `OffshoreUserInfo`. Admins can log out directly from this page.
- **`UserDetails.jsx`** –
This page shows a complete user journey based on the selected onboarding ID:
 - For **Freezone Users**, it shows:
 - Full onboarding summary (industry, activity, visa, office, share capital, etc.)
 - Shareholder details (capital, shares, nationality, etc.)
 - Uploaded documents with download links

- Toggle buttons to mark approval status: visa, license, and trade name
- For **Mainland Users**, it shows:
 - Business name, legal structure, full name, contact info, employee estimate, proposal, and notes
- For **Offshore Users**, it shows:
 - Jurisdiction, contact info, estimated capital, preferred language, proposal, and notes

Admins can toggle approvals in real time and navigate back to the dashboard or logout from the page.

5.2 Key Components

- `navbar.jsx`: Top navigation bar
- `footer.jsx`: Sticky footer
- `chatbot.jsx`: AI-powered assistant using backend API
- `stepsidebar.jsx`: Tracks progress across onboarding steps
- `stepsidebar_freezone.jsx` / `stepsidebar_offshore.jsx`: Customized sidebars for zone type

5.3 Supabase Integration Points

- **Authentication**: User login/signup handled using `supabaseClient.js`
- **Form Data Submission**: On each step, form data is pushed to Supabase tables
- **Admin Access**: Admin dashboard reads user progress and document uploads and verify uploaded documents.

5.4 Chatbot Integration

- `chatbot.jsx` sends queries to FastAPI endpoint (`/chat`)
- Handles user messages and displays Gemini-generated responses
- Useful for answering legal, visa, and business setup queries in real-time

6. Backend Application

The backend is built using **FastAPI**, serving as the engine for both the **AI chatbot** and any backend logic that interacts with Supabase and LanceDB. It follows a modular structure for maintainability and scalability.

6.1 FastAPI Structure

- `main.py`: Application entry point, initializes FastAPI app with CORS settings.
- `api/routes.py`: Defines the `/chat` API endpoint that handles chatbot requests.
- `services/rag_engine.py`: Core logic for Retrieval-Augmented Generation (RAG).
- `supabase/supabase_client.py`: Wrapper for interacting with Supabase backend.
- `db/lancedb/`: Contains vector index (`example_collection.lance`) for similarity search.

The code is structured to keep routing, logic, and services decoupled for better organization.

6.2 RAG Chatbot Engine (`rag_engine.py`)

1. **Input Query**: Accepts user question.
2. **Rephrasing**: Converts it into a stand-alone question using Gemini.
3. **Document Retrieval**: Queries LanceDB for semantically similar documents.
4. **LLM Prompting**: Sends retrieved context + original query to Gemini.
5. **Response Formatting**: Returns AI-generated answer to frontend.

This enables **context-aware answers** without training a custom model.

6.3 Supabase Client Integration

- `Supabase_client.py` connects to Supabase using secure environment variables.
- Used for:
 - Inserting or updating onboarding data.
 - Retrieving user-related business information for dashboards or admin use.
- Designed to be reused across future backend services (e.g., file uploads, status tracking).

7. Supabase Database Design

Supabase is used as the primary backend-as-a-service for **authentication**, **data storage**, and **real-time updates** throughout the onboarding process.

7.1 Tables & Schemas

The following are the primary tables defined in the Supabase PostgreSQL database:

`auth.users`

- Supabase's default authentication table.
- Stores core credentials (email, encrypted password, created_at).

- Related to `Users` table via `auth_user_id`.

Users

- Extends the authenticated user profile.
- Contains fields: `user_id`, `first_name`, `last_name`, `email`, `role`, and `auth_user_id` (FK to `auth.users.id`).

Onboarding

- Central table capturing the onboarding progress of each user.
- Contains fields such as `industry`, `activity`, `freezone`, `ownership`, `visa_requirement`, `trade_name`, `submitted`, `approval status`, `share details`, etc.
- Linked to the `Users` table via `user_id`.

MainlandUserInfo, OffshoreUserInfo

- Tables that store additional business-specific data depending on the type of company being registered.
- Include fields like `contact_name`, `phone_number`, `jurisdiction`, `estimated_capital`, `business_proposal`.

Shareholder

- Captures shareholder details such as name, nationality, email, passport number, capital contribution, and share percentages.
- Linked to `Onboarding` via `onboarding_id`.

Documents

- Tracks uploaded documents.

- Fields include `name`, `type`, `url`, `status`, and foreign keys `onboarding_id` and `shareholder_id`.

Industry, Activities, Freezones, Owner_structure

- Lookup/reference tables used to normalize data.
- Used to populate selection options and maintain data consistency.

Admins (Custom Table)

- Table to store admin metadata such as `email`, `name`, and password for role-based dashboard access.

7.2 Usage in the Application

The Supabase tables are used dynamically throughout the application in the following ways:

- **Create:**
Data is inserted progressively as users fill out each onboarding step. The app handles step-wise creation of records like business info, shareholder details, and document uploads.
- **Read:**
On each step revisit, or when accessing the dashboard, the frontend queries Supabase to fetch and display the latest state of onboarding data, documents, and approvals.
- **Update:**
Users can return to previous steps and revise information. Supabase updates the corresponding fields in real-time.
- **Delete:**
Optional functionality — data cleanup or reset by the user or an admin, typically before final submission.
- **Interaction:**
All database actions are securely handled using the `supabaseClient.js`, which

interfaces with Supabase via auto-generated RESTful endpoints or the client SDK.

8. Chatbot Architecture

The chatbot in Octowize acts as an **AI business assistant**, helping users understand complex legal terms, visa requirements, and company formation options in the UAE. It is built using a **Retrieval-Augmented Generation (RAG)** approach powered by Gemini API.

8.1 Gemini + PgVector Flow

- 1. User Query**
A question is submitted from the frontend via the `/chat` endpoint.
 - 2. Standalone Question Generation**
The input is rephrased (if needed) using Gemini to ensure it is self-contained.
 - 3. Context Retrieval from PgVector**
The rephrased query is semantically matched against pre-indexed documents using a manually created retriever.
 - 4. Prompt Construction**
Relevant document chunks are combined with the query into a single prompt.
 - 5. LLM Response via Gemini API**
Gemini processes the prompt and returns a natural language response.
 - 6. Frontend Display**
The response is streamed or returned in full to the chatbot UI component.
-

8.2 Current Limitations

- **Static Knowledge Base**
All vector search is based on preloaded Freezone/Company setup content.
- **Token Limit**
Gemini's input length restricts the amount of context that can be retrieved and sent at

once.

8.3 Future Scope

- Integrate **PDF ingestion and parsing**
- Add a **feedback loop** for chatbot answers
- Support **zone comparison tables** dynamically within responses

9. Authentication & Authorization

Authentication and user management in Octowize is fully handled by **Supabase Auth**, enabling secure access to onboarding flows and admin dashboards.

9.1 Authentication

- **User Registration & Login**
 - Email/password-based signup
 - Session tokens managed automatically by Supabase
 - Auth status checked before allowing access to onboarding routes
- **Frontend Integration**
 - Handled via `supabaseClient.js` using the Supabase JS SDK
 - Login state persists using secure tokens in local/session storage
- **Backend Access**
 - JWT tokens can be verified in backend if needed (not implemented yet)

9.2 Role-Based Access

- **Regular Users**
 - Access onboarding flow (Freezone, Mainland, Offshore)
 - Can view, update, and submit business-related forms
- **Admins**
 - Access admin dashboard
 - View user submissions
 - Monitor document uploads and onboarding progress

Roles are determined by a `role` field in the Supabase `users` table.